

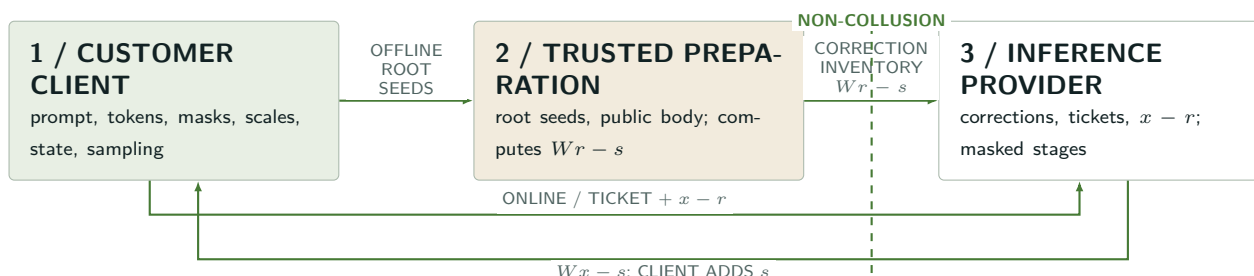
Private LLM inference, prepared before the prompt

CURRENT DESIGN

Public-weight runtime

PLLM v0.17.0a1

PLLM separates public-model computation across a customer client, trusted preparation service, and inference provider. Fresh one-time inventory moves preparation off the online path without removing its cost or trust assumptions.



What PLLM changes

Ordinary hosted inference exposes prompts and generated text to one provider. PLLM instead keeps tokenization, nonlinear operations, attention state, sampling, and decoding in the customer environment. Public transformer-body matrices run remotely as masked integer projections. This is a deployment boundary, not a claim that an unmodified model API becomes private by changing its URL.^[1]

Prepared inventory

Before chat, the client commits an inventory to one model, body, stage set, quantization profile, and attempt budget. It sends one domain-separated root seed per stage to preparation. Preparation expands one-time input masks r , output masks s , and tickets, computes $Wr - s$, then pushes each batch to inference. Inference durably acknowledges every stage and seals the inventory READY. Online, it atomically consumes a ticket and returns

$$W(x - r) + (Wr - s) = Wx - s.$$

The client adds s . Prefill rows share a compact matrix envelope; decode retains one ticket per stage on a persistent connection. Unused reserved rows burn on cancellation or failure. Inventory is memory-only and disappears on restart or idle expiry.^[1]

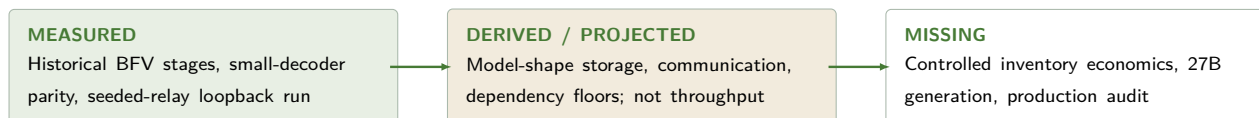
Boundary, not invisibility

Protected from inference: plaintext prompts and outputs, token identities, root seeds, masks, activation scales, local attention and recurrent state, and sampling choices. Preparation sees seeds and bound stage metadata, but not online masked activations. Inference sees corrections and masked activations, but not seeds.

Still exposed: both services know the public model, stage names, tensor shapes, quantization profile, scheduling, timing, traffic volume, and approximate sequence lengths. Preparation must follow the protocol, erase expanded masks, and not collude with inference. Two services under one untrusted operator do not meet this assumption; authentication and TLS protect channels but do not prove independence, erasure, or correct model execution.^[2]

Fresh uniform masks hide each online integer activation from inference when viewed alone. That claim is narrower than end-to-end confidential computing: it assumes protocol-conforming roles, keeps nonlinear work and state on the client, and does not verify provider execution. A network observer able to join both channels, a preparation service retaining masks, or colluding services can recover activation information. Deployment review must therefore name operators and log-access paths, not merely count endpoints.

Price the whole lifecycle, then decide.



Economic cost ledger

No repository evidence supports an ROI or savings percentage. A deployment model should count:

- **Client:** local tokenization, nonlinear math, KV/recurrent state, public boundary matrices, memory, and model-plan transfer.
- **Preparation:** duplicate public body storage, mask expansion, matrix work, correction egress, secure erasure, and idle capacity.
- **Inference:** body storage, masked projection compute, inventory memory, online ingress/egress, and ticket bookkeeping.
- **Operations:** three credentials, TLS endpoints, telemetry, upgrades, replenishment, failures, and independent control of preparation.

Unit economics therefore depend on prompt shape, generated tokens, stage dimensions, concurrent demand, refill batch, burn rate, bandwidth price, and required operator separation.

Potential impact

Offline preparation removes preparation round trips from time-to-token, but does not remove preparation compute or correction traffic. Batching can amortize fixed work while increasing stranded or burned inventory. Compact prefill transport and geometric KV allocation reduce serialization and client-copy pressure; neither proves lower cloud spend. Self-hosting preparation can simplify trust ownership while moving hardware and operations to the customer.

Evidence ladder

MEASURED. The historical lifecycle study measured a 768-by-256 BFV preparation stage at 0.633 seconds for 2,048 correlations versus 3.122 seconds for its matched control, a 4.93x speedup. A trained small decoder produced 96 tokens matching its clear quantized target. A separate 8 September loopback run used Qwen2.5-0.5B, three processes, and the earlier just-in-time seeded relay: 32 tokens in 6.07 seconds. These results do not benchmark current offline inventory.^{[3][4]}

DERIVED. The same study calculated 48.22 GiB for 2,048 raw Qwen3.5-27B mask sets and a 1.34 token/s ceiling on an aggregate 1 Gbit/s link under stated occupancy assumptions. These are capacity models, not measured 27B generation or present-runtime economics.^[3]

MISSING. Production non-collusion and malicious-provider review; controlled repeated cold/warm preparation, latency, throughput, burn-rate, and dollar-cost studies; full-checkpoint 27B generation; GPU, energy, failure-recovery, and multi-tenant studies.

Decision gates

1. **Trust:** preparation has independent control or sits inside the customer boundary; exposed metadata is acceptable.
2. **Correctness:** target checkpoint, context, quantization, and output quality pass workload-specific validation.
3. **Economics:** measured cold and warm ledgers beat explicit latency, capacity, and cost ceilings, including burned rows.
4. **Operations:** credential separation, TLS, erasure, observability, restart behavior, and incident response survive a pilot.

Proceed only when all four gates have named owners and recorded evidence.

References

- [1] PLLM ARCHITECTURE.md, v0.17.0a1 source distribution, 11 Sep 2026.
- [2] PLLM SECURITY.md and website Security page, v0.17.0a1 source distribution.
- [3] PLLM research/evidence/CLAIMS.json and STUDY_REPORT.md, historical evidence snapshot.
- [4] PLLM research/evidence/qwen-prepared-inference.json, 8 Sep 2026; loopback and pre-inventory.